Final:

Q. What is weight and bias?

⇒ **Weight:** is determines the importance of an input in a neural network.

- Each input has a weight.
- Higher weight = greater influence on output.
- Weights are learned during training.

Example: If $x=2$ and $w=0.5$, then

$$\text{contribution} = 2 \times 0.5 = 1$$

Bias: is an additional value added to the weighted sum to adjust the output.

- Helps the model fit data better.
- Similar to the intercept in a linear equation.
- Allows output shifting independent of inputs.

formula: $z = \sum w_i x_i + b$

Q. What is forward propagation?

⇒ **Forward Propagation:-** is the process of passing input data through a neural network to calculate the output.

- Data flows from input layer → hidden layer(s) → output layer.
- Uses weights and biases to compute outputs.
- Activation functions are applied at each neuron.
- It is the first step in training and prediction.

Example:

Input image → Neural network → Predicted label (Cat/Dog)

*Stemming:

is a text preprocessing technique that reduces words to their root form (stem). (removing suffix and prefixes).

- Used in NLP.
- Helps reduces vocabulary size.
- Different forms of a word are treated as the same word.

Examples: Playing → play
Played → play
playing, plays, played → play.

Q. Problems in stemming.-

⇒ • May generate non-dictionary words.
- Can remove too many characters (over-stemming)
- can fail to reduce related words to the same stem (under-stemming)
- May loss the original meaning of words.

Example:

University → Univers (incorrect stem)

* Better and good stemming are related but not reduced to the same stem.

Q. Advantanges and disadvantages of stemming.

⇒

Advantages	Disadvantages
1. Reduces vocabulary size.	1. May produces meaningless root words.
2. Improves text processing speed.	2. Can cause over-stemming
3. Saves storage and memory	3. Can cause under-stemming
4. Helps group similar words together.	4. May lose the original meaning of words.
5. Useful for search engines and NLP tasks	5. Reduces accuracy in some applications.

Q. What is NLP?

⇒ Natural Language Processing is a branch of AI that enables computers to understand, process, and generate human language. It helps machines interact with text and speech data.

Q. NLP Applications

⇒ NLP used in

- chatbots,
- machine translation.
- sentiment analysis.
- speech recognition.
- spam detection.

It helps automate language-related tasks.

Q. Whats Tokenization?

⇒ is the process of breaking text into smaller units called tokens, such as words or sentences. It is usually the first step in NLP preprocessing.

Q. Whats Stop Word Removal?

⇒ is the process of removing common words such as "the", "is", "and" from text. This helps reduce unnecessary data and improve processing efficiency.

Q. Can we always use stop word removal before stemming?

⇒ No, stop word removal is not always necessary before stemming. In some tasks, stop words carry important information and removing them may reduce accuracy.

Q. Whats Lemmatization?

⇒ is the process of converting words to their dictionary base form (lemma) using vocabulary and grammer rules. For example. running becomes run and better becomes good.

Q. Differences between Lemmatization and Stemming.

⇒

Stemming	Lemmatization
1. Removes prefixes/suffixes to get the root word.	1. Converts a word to its dictionary based form (lemma)
2. Uses simple rule-based cutting.	2. Uses vocabulary and grammertical analysis.
3. Faster and computationally cheaper.	3. Slower and more computationally expensive.
4. May produce non-meaningful words.	4. Produces meaningful words.
5. Less accurate.	5. More accurate.
6. Ex: Studies → Studi	6. Ex: Studies → Study.

Q. Advantages and disadvantages of both.

⇒ ~~Advan~~ Stemming:

Advantages	Disadvantages
1. Faster and computationally efficient.	1. May produce meaningless words.
2. Reduces vocabulary size for faster processing.	2. Less accurate than Lemmatization

Lemmatization:

Advantages	Disadvantages
1. Produces meaningfull dictionary words.	1. Slower than stemming.
2. More accurate and preserves word meaning.	2. Requires more computational resources.

Q. Definition of corpus, vocabulary, document, word.

⇒ Corpus: is a collection of documents or text data used for NLP tasks. It serves as the dataset for training and analyzing language models.

Vocabulary: is the set of all unique words present in a corpus. It represents the words that a model can recognize or process.

Document: is a single unit of text within a corpus, such as an article, email, or review. A corpus consists of one or more documents.

Word: is the smallest meaningfull unit of text used in NLP. Words are typically obtined after tokenization and form the building blocks of documents.

- One hot Encoding:

is a technique used to convert categorical data or words into binary vectors. In the vector, only one person is 1 and all other positions are 0.

Example: Vocabulary = {cat, Dog, Bird}

- cat ⇒ [1, 0, 0]
- Dog ⇒ [0, 1, 0]
- Bird ⇒ [0, 0, 1]

*

D1	Student read DL
D2	DL read DL
D3	Student write comment
D4	DL write comment.

Corpus: Student read DL
DL read DL
Student write comment
DL write comment

Vocabulary: Student, read DL write comment

D=5

D1 Vector: [1,0,0,0,0], [0,1,0,0,0], [0,0,1,0,0]

Diemension: [3,5] vocabulary = 5

Q. Advantages and disadvantages of One-hot-encoding.

⇒

Advantages	Disadvantages
1. Simple and easy to implement.	1. Creates high-dimentional vectors for large vocabularies
2. No numerical relationship is assumed between categories	2. Produces sparse vectors. (mostly zeros)
3. Works well for basic machine learning models.	3. Cannot capture semantic relationships between words

Bag of Words (BoW)

⇒ is a text representation technique that converts text into numerical vectors based on the frequency of words. It ignores grammer and word order, considering only the occurance of words.

D1	Student read DL	1
D2	DL read DL But	1
D3	Student write comment	0
D4	DL DL write DL comment DL	0

Vocabulary: Student read DL write comment But

Doc	Student	read	DL	write	comment	But
D1	1	1	1	0	0	0
D4	0	0	4	1	1	0

Q. Advantages and disadvantages of BoW.

⇒

Advantages	Disadvantages
1. Simple and easy to implement.	1. Ignores word order and context.
2. Converts text into numerical form for machine learning.	2. Creates high-dimension feature vectors.
3. Fast to train and process.	3. Produces sparse matrices

	with many zeros
4. Effective for basic text classification tasks.	4. Cannot capture semantic meaning between words.

TF-IDF:

is a text representation technique that measures how important a word is in a document relative to a collection of documents (corpus). It gives higher weight to important words and lower weight to very common words.

- Term frequency
- Inverse Document Frequency

$$TF(t,d) = \frac{\text{Number of times term } t \text{ appears in document } d}{\text{Total number of terms in document } d}$$

$$IDF(t) = \log_e \frac{\text{total number of documents in the corpus}}{\text{Number of documents with term } t \text{ in them.}}$$

D1:	Student watch mobile	1
D2:	Mobile watch ~~student~~ Screen	1
D3:	Student write comment	0
D4:	Screen write comment.	0

TF table:

Term	D1	D2	D3	D4
Student	1/3	0	1/3	0
Watch	1/3	1/3	0	0
Mobile	1/3	1/3	0	0
Screen	0	1/3	0	1/3
Write	0	0	1/3	1/3
Comment	0	0	1/3	1/3

Vocabulary:

Student, Watch, Mobile, Screen, write, comment

TF-IDF table: Total Doc, N=4, Each term appears in 2 documen

Term	D1	D2	D3	D4
Student	$1/3 \log \frac{4}{2}$	0	$1/3 \log 4/3$	0
Watch	$1/3 \log 4/2$	$1/3 \log 4/2$	0	0
Mobile	$1/3 \log 4/2$	$1/3 \log 4/2$	0	0
Screen	0	$1/3 \log 4/2$	0	$1/3 \log 4/2$
Write	0	0	$1/3 \log 4/2$	$1/3 \log 4/2$
Comment	0	0	$1/3 \log 4/2$	$1/3 \log 4/2$

Word Embedding:

is a technique that converts words into dense numerical vectors while preserving their semantic meaning. Similar words are represented by vectors that are close to each other in the vector space.

Example: Words like "King" and "Queen" will have similar vector representations because they are semantically related.

Types of Word Embedding

- Frequency Based
  - One hot encoding
  - Bag of Words (BoW)
  - TF-IDF
- Prediction Based
  - Word2Vec
    - CBoW (Continuous BoW)
    - Skip-Gram
  - Glove
  - FastText.

Q. Why we need word embedding?

⇒ Word embedding is needed to convert words into meaningful numerical vectors that computers can process. It captures the semantic relationships between words, unlike One Hot Encoding or BoW.

Key points:

- Reduces high-dimensional sparse vectors.
- Captures word meaning and context.
- Improves performance of NLP and DL models.

Q. Advantages and disadvantages of TF-IDF.

⇒

Advantages	Disadvantages
1. Highlights important words in a document.	1. Ignores word order and content.
2. Reduces the impact of common words.	2. Cannot capture semantic meaning.
3. Simple and easy to implement	3. Creates high-dimentional sparse vectors.
4. Works well for text classification and information retrieval.	4. Performance decreases with very large vocabularies.

	King	Queen	Man	Woman	Monkey
Gender	1	0	1	0	1
Wealth	1	1	0.3	0.3	0
Power	1	0.7	0.2	0.2	0
Weight	0.8	0.4	0.6	0.5	0.3
Speak	1	1	1	1	0

***Word2Vec:** Man : [0.4 , 0.5]

Woman : [0.5 , 0.6]

Queen : [0.5 , 0.62]

King [1 , 1 , 1, 0.8 , 1]

Queen [0, 1, 0.7, 0.4, 1]

Man [1, 0.3, 0.2, 0.6, 1]

Woman [0, 0.3, 0.2, 0.5, 1]

Monkey [1, 0, 0, 0.3, 0]

King − Man + Woma : [0 1 1 0.7 1]

Almost Queen Embedding

Cosine Similarity:

is a measure used to determine how similar two vectors on documents are by calculating the cosine of the angle between them. Its value ranges from −1 to 1, where 1 means identical and 0 means no similarity.

Formula:

$$\text{Cosine Similarity}(A, B) = \frac{A \cdot B}{|A| \cdot |B|}$$

Applications

- Document Similarity
- Search engines
- Recommendation system

Steps to work:

1. create TF table + TF-IDF table
2. Queary column add.
3. Use cosine similarity theorem.
4. Rank Documents.

Example:

(i) Student watch mobile.

(ii) Mobile watch screen.

(iii) Student write comment.

(iv) Screen write comment.

for this sentence:

"Student watch watch mobile screen"

find cosine similarity

⇒ TF-table: total documents = 4

Vocabulary = {Student, watch, mobile, screen, write, comment}

Term	Doc 1	Doc 2	Doc 3	Doc 4	Q	$IDF(t) = \log\left(\frac{N}{df}\right)$
Student	1/3	0	1/3	0	1	$\log \frac{4}{2} = 0.3$
Watch	1/3	1/3	0	0	2	$\log \frac{4}{2} = 0.3$
Mobile	1/3	1/3	0	0	1	$\log \frac{4}{2} = 0.3$
Screen	0	0	1/3	1/3	0	$\log \frac{4}{2} = 0.3$
Write	0	1/3	0	1/3	1	$\log \frac{4}{2} = 0.3$
Comment	0	0	1/3	1/3	0	$\log \frac{4}{2} = 0.3$

TF-IDF table

Term	Doc 1	Doc 2	Doc 3	Doc 4	Q
Student	$1/3 \times \log \frac{4}{2} = 0.1$	0	0.1	0	$1 \times 0.3 = 0.3$
watch	0.1	0.1	0	0	0.6
mobile	0.1	0.1	0	0	0.3
write	0	0	0.1	0.1	0
screen	0	0.1	0	0.1	0.3
comment	0	0	0.1	0.1	0

Cosine Similarity $= \frac{D \cdot Q}{|D| \cdot |Q|}$

$|Q| = \sqrt{(0.3)^2 + (0.6)^2 + (0.3)^2 + (0.3)^2} = 0.79$

$|D1| = \sqrt{(0.1)^2 + (0.1)^2 + (0.1)^2} = 0.17$

$|D2| = \sqrt{(0.1)^2 + (0.1)^2 + (0.1)^2} = 0.17$

$|D3| = \sqrt{(0.1)^2 + (0.1)^2 + (0.1)^2} = 0.17$

$|D4| = \sqrt{(0.1)^2 + (0.1)^2 + (0.1)^2} = 0.17$

$\therefore CS(Q, D1) = \frac{0.1 \times 0.3 + 0.1 \times 0.3 + 0.1 \times 0.3}{0.17 \times 0.79} = 0.89$

$\therefore CS(Q, D2) = \frac{0.1 \times 0.6 + 0.1 \times 0.3 + 0.1 \times 0.3}{0.17 \times 0.79} = 0.89$

$\therefore CS(Q, D3) = 0.22$

$\therefore CS(Q, D4) = 0.22$

$\therefore$ D1 and D2 most relevant. **Answer**

Continuas Bag of Words

is a Word2Vec model that predicts a target using its surronding context words. It learns word embeddings by analyzing neighboring words.

Examples

Sentence : "I love deep learning."

Context : I, love, learning → Predict deep

Key points:

- Predicts the center word from context words.
- Faster to train than skip-Gram.
- Works well with frequent words.

Fake/dummy problem → solution

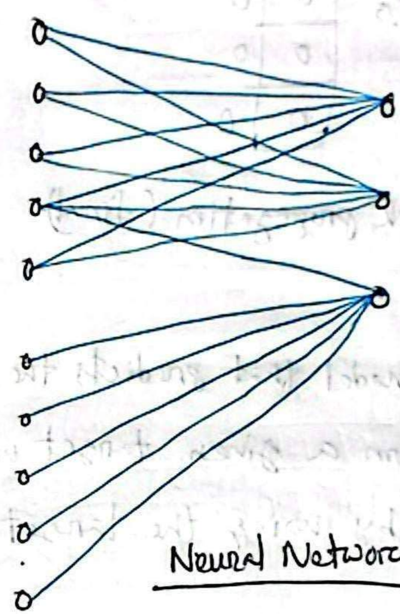
Attend class for deep learning

target

content

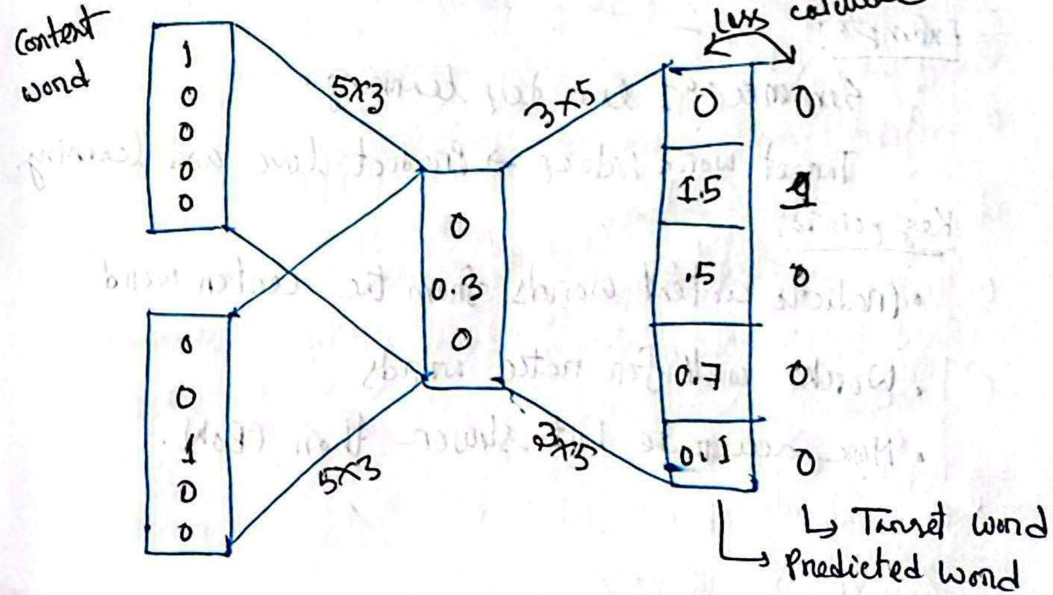
content word		target word
Attend, for	⟶	Class
Class, deep	⟶	for
for, learning	⟶	deep.

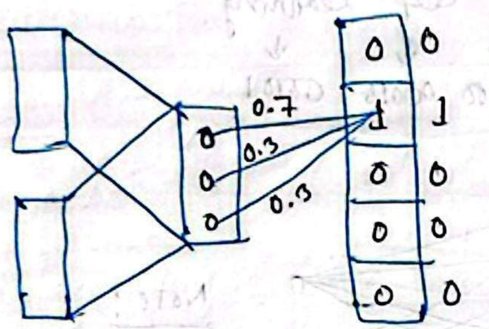
	Attend	class	for	deep	learning.
1hot encoding	10000	01000	00100	00010	00001



Neural Network

Note: three point here because it three feature available





After forward-back propagation (final)

Skip-Gram:

is a Word2Vec model that predicts the surrounding context words from a given target word. It learns word embeddings by using the target word to predict its neighbors.

Example:

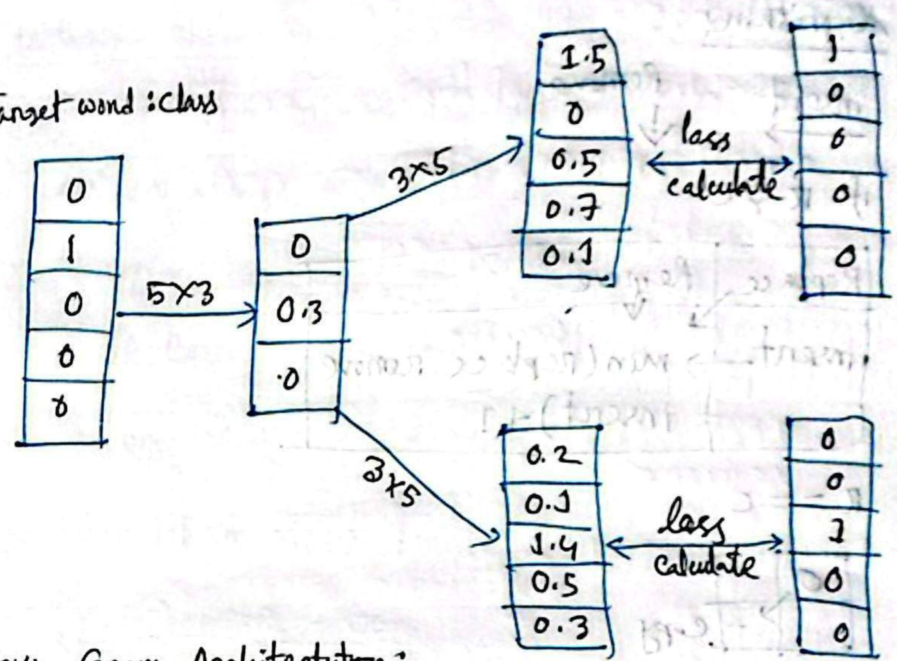
Sentence: "I love deep learning"

Target word: deep → Predict love and learning.

Key points:

- Predicts context words from the center word.
- Works well for rare words
- More accurate but slower than CBoW.

Target word: class



Skip-Gram Architecture:

Minimum Edit Distance (MED)

is the minimum number of operation required to transform one string into another. The allowed operations are insertion, deletion, substitution.

Example:

"cat" → "cut"

Replace a with u.

MED = 1

Applications:

- Spell checking
- Auto correction
- NLP text matching

Algorithm:

insert $\rightarrow$ Remove $\downarrow$

if $r \neq c$

Replace	Remove
insert	$\rightarrow \min(\text{replace}, \text{remove}, \text{insert}) + 1$

$r == c$

0	
	copy

Example: adceg $\rightarrow$ abcfg

	Null	a	b	c	f	g
Null	0	1	2	3	4	5
a	1	0	1	2	3	4
d	2	1	1	2	3	4
c	3	2	2	1	2	3
e	4	3	3	2	2	3
g	5	4	4	3	3	2

Answer is 2.

* **Translate:**

is the automatic conversion of text from one language to another while preserving its meaning.

Ex. (i) Think about it → এটা নিয়ে চিন্তা কর

(ii) Come in → ভিতরে আসুন।

$\boxed{\text{encoder}} \rightarrow \boxed{\text{decoder}}$

↑

think about it

Seq2Seq model:

is a deep learning model that converts one sequence into another sequence. It consists of an Encoder and a Decoder, where the encoder processes the input sequence and the decoder generates the output sequence.

Ex: Eng: "I am a student."

↓

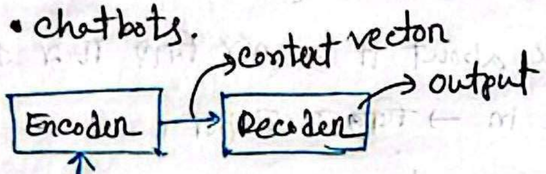
Seq 2 Seq model

↓

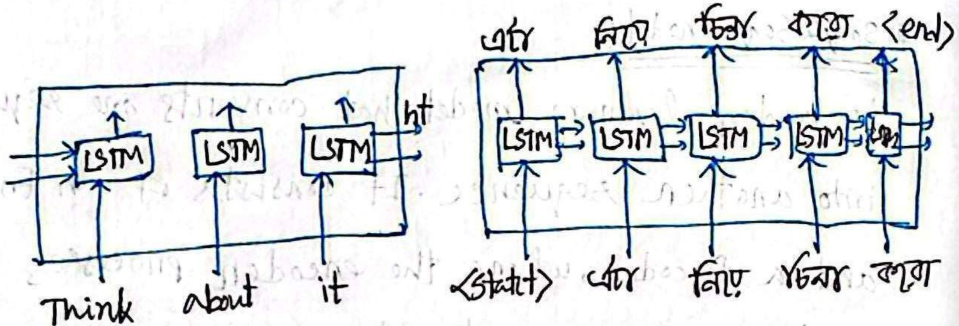
Ban: "আমি একজন ছাত্র।"

Applications:

- Machine translation
- Text Summarization
- chatbots.



Encoder consist a lot of LSTM model, decoder also.



Encoder side Decoder side

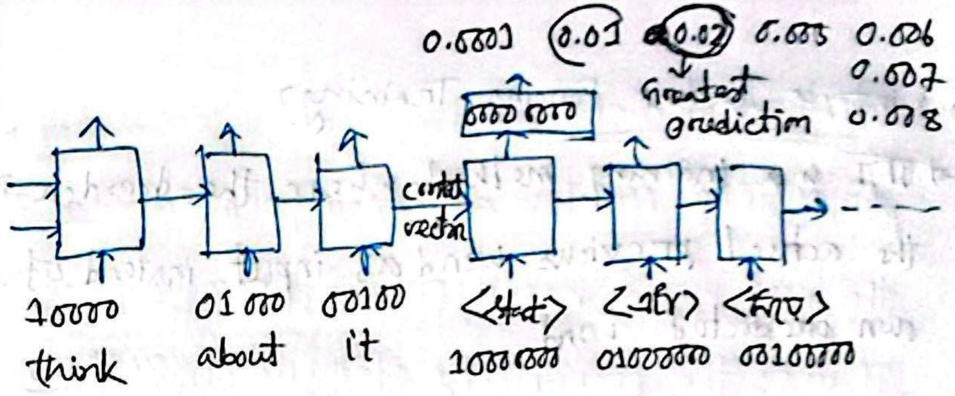
*Training phase 1

1 Think 2 about 3 it → 1 <start> 2 এটা 3 নিয়ে 4 চিন্তা 5 করো

4 come 5 in → 6 [illegible] 7 [illegible] 8 <end>

Think 1-hot encoding → 10000

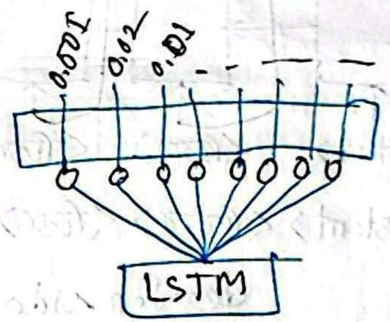
[illegible] " " → 00010000



Encoder Decoder

$y = [0100000], [0010000] \cdots [0000001]$

$\hat{y} = [0.001, 0.01, \ldots] [0.1, \ldots] \cdots [0.1 \ldots]$



Forward

Loss calculate → Back Propagation

↓

optimizer

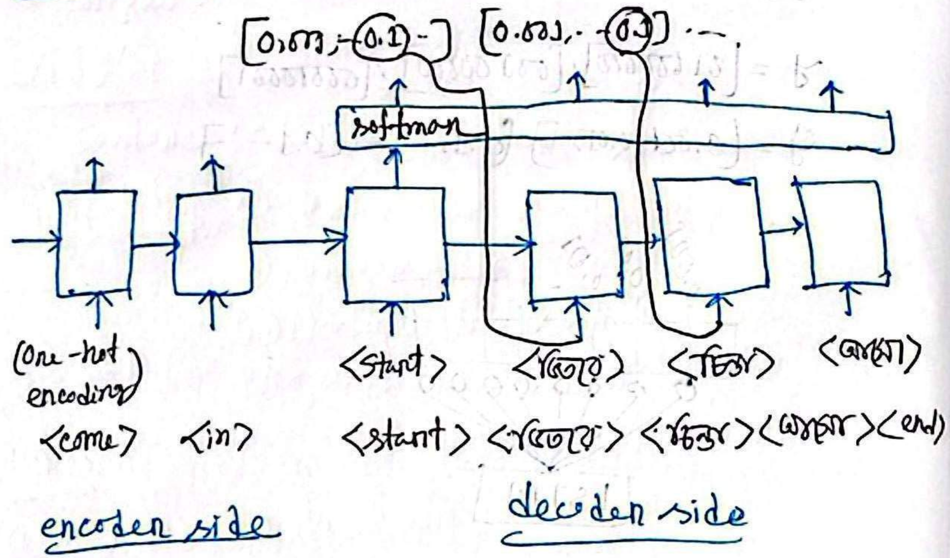
↓

A.F, W, B update

Q. What is Teacher Forcing Training?

⇒ TFT is a training method where the decoder receives the actual previous word as input instead of its own predicted word.

* Testing Phase:



Q. How to improve seq2seq model?

⇒ The basic seq2seq model struggles with long sentences because the entire input must be compressed into a single content vector. Several techniques are used to improve its performance.

1. Attention Mechanism:
   - Allows the decoder to focus on important parts of the sentence.
   - Solves the fixed content vector problem.

2. Bidirectional LSTM:
   - Reads the input sequence in both forward and backward directions.
   - Captures more content.

3. Beam Search:
   - Considers multiple possible outputs instead of choosing only the most probable word at each step.
   - Produces better translations.

4. Transformer Model:
   - Replaces LSTMs with self-attention.
   - Faster and more accurate than traditional Seq2Seq.

N-Gram Model:

An N-Gram Model is a language model that predicts the next word based on the previous N-1 words. It uses the probability of word sequences from a corpus.

~~Example:~~

~~Sentence: "I Love Deep Learning."~~

Applications: (i) Next-word prediction
(ii) Autocomplete
(iii) Machine translation.
(iv) Speech recognition.

Types:

(i) Unigrams → no history used
(ii) Bigrams → One history used
(iii) Trigram → Two history used.

Example:

Sentence: "I Love Deep Learning."

- Unigram: {I, love, Deep, Learning}
- Bigram: {I, love, Love Deep, Deep Learning}
- Trigram: {I love Deep, Love Deep learning}

conditional probability is the probability of an event occurring given that another event has already occurred. In NLP, it is used to predict a word based on previous words.

formula: $p(B|A) = \frac{p(A,B)}{p(A)}$

$$p(A,B) = p(A) \cdot p(B|A)$$

$$p(A,B,C,D) = p(A) \cdot p(B|A) \cdot p(C|A,B) \cdot p(D|A,B,C)$$

$$p(x_1,x_2,x_3 \ldots x_n) = p(x_1) \cdot p(x_2|x_1) \cdot p(x_3|x_1,x_2) \ldots p(x_n|x_1 \ldots x_{n-1})$$

$$p(\text{Think about it}) = p(\text{Think}) \cdot p(\text{about}|\text{Think}) \cdot p(\text{this}|\text{Think, about})$$

Example:

"about five min from ___ "

Unigram: no word history.

$$p(w_i) = \prod_{i=1}^{n} p(w_i) = \frac{count(w_i)}{N}$$

$$p(\text{college}|\text{about, five, min, from}) = p(\text{college}) = \frac{count(\text{college})}{N}$$

$$p(\text{home}|\text{about, five, min, from}) = p(\text{home}) = \frac{count(\text{home})}{N}$$

Unigram chain rule:

count(about, five, min, from) = p(about) × p(five) × p(min) × p(from)

count(about, five, min, from, college) = p(about) p(five) p(min) p(from) p(college)

count(about, five, min, from, home) = p(about) p(five) p(min) p(from) p(home)

Bigram: One word history.

$$p(w_1, w_2) = \prod_{i=1}^{n} p(w_2|w_1) \rightarrow p(w_i|w_{i-1}) = \frac{count(w_{i-1}, w_i)}{count(w_{i-1})}$$

$$p(college|about, five, min, from) = \frac{count(about, five, min, from, college)}{count(about, five, min, from)}$$

$$p(home|about, five, min, from) = \frac{count(about, five, min, from, home)}{count(about, five, min, from)}$$

$$count(about\ five\ min\ from) = p(about|\langle s\rangle) \cdot p(five|about) \cdot p(min|five) \cdot p(from|min)$$

$$count(about\ five\ min\ from\ college) = p(about|\langle s\rangle)\ p(five|about)\ p(min|five) \cdot p(from|min)\ p(college|from)$$

$$count(about\ five\ min\ from\ home) = p(about|\langle s\rangle)\ p(five|about)\ p(min|five) \cdot p(from|min)\ p(home|from)$$

Trigram: Two word history:

$$p(w_1, w_2, w_3) = \prod_{i=1}^{n} p(w_i|w_{i-2}, w_{i-1}) = \frac{count(w_{i-2}, w_{i-1}, w_i)}{count(w_{i-2}, w_{i-1})}$$

$$p(college|about, five, min, from) = \frac{count(min, from, college)}{count(min, from)}$$

$$p(home|about, five, min, from) = \frac{count(min, from, home)}{count(min, from)}$$

$$\cancel{count}\ p(about, five, min, from) = p(about|\langle s\rangle, \langle s\rangle) \times p(five|\langle s\rangle, about) \times p(min|about, five) \times p(from|five, min)$$

$$p(about, five, min, from, college) = p(about, five, min, from) \times p(college|min, from)$$

$$p(about, five, min, from, college) = p(about, five, min, from) \times p(home|min, from)$$

Example:

Given corpus:

<s> I am Henry </s>
<s> I like college </s>
<s> Do Henry like college </s>
<s> Henry I am </s>
<s> Do I like Henry </s>
<s> Do I like college </s>
<s> do like Henry </s>

Q: <s> Do ?

$w_{i-1} = Do$

using bigram.

Soln:

Word	Frequency
<s>	7
</s>	7
I	5
am	2
Henry	5
like	5
college	3
do	4

Next word	Probability Next word $= \frac{count(w_{i-1}, w_i)}{count(w_{i-1})}$
p(</s>|do)	0/4
p(I|do)	2/4
p(am|do)	0/4
p(Henry|do)	1/4
p(like|do)	1/4
p(college|do)	0/4
p(do|do)	0/4

I is the most preferable next word.

unigram:

Next word	$P.N.W = \frac{count(w_i)}{N}$
p(<s>)	7/38
p(I)	5/38
p(am)	2/38
p(Henry)	5/38
p(like)	5/38
p(college)	3/38
p(do)	4/38

N = total number of tokens.

$= 7+7+5+2+5+5+3+4$

$= 38$

~~I will be most preferable. <s> is structural tags that why this will not count.~~ Ties between "I", "Henry", "Like".

Trigram:

Next word	$P.N.W = \frac{count(w_{i-2}, w_{i-1}, w_i)}{count(w_{i-2}, w_{i-1})}$
p(</s>|<s>,do)	0/4
p(I|<s>,do)	2/4
p(am|<s>,do)	0/4
p(Henry|<s>,do)	1/4
p(like|<s>,do)	1/4
p(college|<s>,do)	0/4
p(do|<s>,do)	0/4

I most preferable.